Web Design and Development

Review Questions & Solutions Manual

Section 1: Pages 35–36 Solutions

A. Multiple Choice Questions

#	Question Summary	Correct Answer
1	Relationship between Internet and Web	**d)** The Web is an information service that runs on the Internet.
2	Uses Internet but not the Web	**a)** Sending email from a desktop email program
3	Main lesson of the browser wars	**b)** Browsers must implement shared, open standards.
4	Domain Name System (DNS) purpose	**c)** Translates domain names into IP addresses
5	Best built as a static website	**c)** A photographer's portfolio
6	Distinguishes web app from informational site	**d)** It lets users perform tasks on their own data.
7	Back-end development task	**b)** Implementing authentication and APIs
8	Client-side vs. server-side rule enforcement	**c)** Client-side code can be bypassed or modified by the user.
9	Order of lifecycle phases	**b)** Requirements, Planning,

		Design, Development, Testing, Deployment, Maintenance
10	Low-fidelity layout sketch without styling	**d)** wireframe
11	Definition of agile development	**d)** Working features are delivered in short, repeated iterations.
12	404 error on css/Styles.css vs css/styles.css	**a)** Correct the capitalization of the file name the page asks for

B. True/False Questions

1. **False** — The Internet and the Web were invented at the same time. *(The Internet was developed in the late 1960s/1970s; the Web was invented by Tim Berners-Lee in 1989).*
2. **True** — Email can be used without the Web.
3. **False** — A static website cannot contain JavaScript. *(Static sites can include client-side JavaScript).*
4. **False** — Checking a form in the browser alone protects it from malicious input.
5. **True** — A wireframe shows a page's layout without its final colors or images.
6. **True** — In the waterfall model, each phase is completed before the next begins.
7. **False** — Edits made in the developer tools change the files on the server. *(DevTools changes are strictly temporary and local to the browser).*
8. **True** — A full-stack developer works across the front end, back end, and data layer.

C. Short Answer Questions

1. Internet vs. World Wide Web

- **Difference:** The **Internet** is the physical and global network infrastructure connecting millions of computers via networking protocols. The **Web** (World Wide Web) is an application service that operates *on top* of the Internet, utilizing the HTTP/HTTPS protocol to transfer hypertext documents.
- **Non-Web Internet Services:**
    - Desktop Email Clients (using SMTP, IMAP, or POP3)
    - Direct File Transfer (via FTP or SSH/SFTP)

2. Standard Web Request Lifecycle

1. **URL Parsing:** The browser parses the entered URL to extract the protocol, domain name, and resource path.
2. **DNS Resolution:** The browser performs a DNS lookup to resolve the domain name into a numerical IP address.
3. **Connection Establishment:** The browser initiates a TCP connection (and TLS handshake if HTTPS) with the server at the resolved IP address.
4. **HTTP Request:** The browser sends an HTTP request message specifying the method, path, and headers.
5. **HTTP Response:** The web server processes the request and responds with a status code and content (typically HTML).
6. **Rendering:** The browser parses the HTML syntax, builds the Document Object Model (DOM), requests secondary resources (CSS, JS, images), and renders the final web page.

3. Static vs. Dynamic Websites

- **Speed:** Static sites are faster because pre-built HTML files are served directly from storage or a CDN. Dynamic sites require computational processing and database calls on every request.
- **Security:** Static sites present a much smaller attack surface. Dynamic sites carry elevated risks such as SQL injection, Cross-Site Scripting (XSS), and authentication exploits.
- **Effort to Update:** Static sites require direct editing of code and redeployment for content updates. Dynamic sites allow easy updates by non-technical users via Content Management Systems (CMS) or databases.

4. Software Development Models: Waterfall vs. Agile

- **Waterfall:** A linear, sequential development approach where every phase (Requirements, Design, Code, Test) must complete entirely before the next starts. Best suited for projects with fixed, unchangeable scope.
- **Agile:** An iterative, flexible approach where product functionality is delivered in short, repeating cycles (sprints). Accommodates evolving specifications and frequent user feedback.

5. Essential Browser Developer Tools Panels

- **Elements:** Used to inspect, manipulate, and live-edit the page's HTML structure and CSS styles.

- **Console:** Used for outputting diagnostic logs, executing live JavaScript code, and inspecting runtime errors.
- **Network:** Monitors all incoming and outgoing network traffic, HTTP status codes, payload sizes, and asset load times.
- **Device Toolbar:** Simulates various mobile and tablet viewports, pixel ratios, and touch events for responsive design testing.

Section 2: Pages 48–50 Solutions

A. Multiple Choice Questions

#	Question Summary	Correct Answer
1	Initiator of HTTP exchange	**a)** The client
2	Meaning of HTTP statelessness	**d)** each request is handled independently of earlier ones
3	Server response to a directory path	**c)** The folder's default document, such as index.html
4	Default port for HTTPS	**d)** 443
5	DNS server hosting source records	**b)** The authoritative name server
6	Reason for DNS propagation delays	**a)** Resolvers cache answers until the TTL expires.
7	Identify query component in URL	**c)** term=2
8	URL encoding character for space	**d)** %20
9	Resolution of courses/web/intro.html + ../index.html	**b)** /courses/index.html

10	Valid HTTP request line syntax	**a)** GET /news.html HTTP/1.1
11	Response code for valid cached content	**c)** 304 Not Modified
12	Server diagnostic for 500 status code	**d)** The server's program and error logs

B. True/False Questions

1. **False** — Over HTTP, a server can send a page that the browser did not request. *(HTTP/1.1 is strictly request-response; servers cannot send unrequested content).*
2. **True** — An IP address identifies a device, and a port identifies a program on it.
3. **False** — A domain name is read from left to right, most general part first. *(Domains are hierarchical from right to left, starting at the TLD).*
4. **False** — The fragment of a URL is sent to the server. *(URL fragments/anchors # are processed entirely by the client/browser).*
5. **True** — A GET request normally has no body.
6. **False** — POST data is encrypted even when sent over HTTP. *(Plain HTTP sends data in plaintext regardless of request method).*
7. **True** — A 4xx status code indicates a problem with the request.
8. **False** — A valid certificate proves that a website is honest. *(Certificates verify domain ownership and enable encryption, not business integrity or legitimacy).*

C. Short Answer Questions

1. Client-Server Architecture & HTTP Statelessness

- **Client-Server Architecture:** A distributed computing structure where the client (browser) requests services or resources, and the server processes and responds to those requests.
- **Importance of Statelessness:** Because the server treats each request as an isolated transaction, it doesn't need to hold memory connections open for millions of concurrent users. This enables massive web scalability. Features requiring state (e.g., shopping carts, active logins) use secondary mechanisms like cookies or session tokens.

2. Steps of Uncached DNS Resolution

1. **Recursive Resolver Request:** The browser sends the query (e.g., example.com) to the Recursive DNS Resolver (typically managed by the ISP or network).
2. **Root Server Query:** The Resolver queries a Root Name Server to locate the appropriate Top-Level Domain (TLD) server.
3. **TLD Server Query:** The Root points the Resolver to the .com TLD Name Server, which is then queried.
4. **Authoritative Server Query:** The TLD server responds with the address of the Authoritative Name Server for example.com.
5. **IP Resolution & Caching:** The Resolver queries the Authoritative server, receives the exact IP address, caches the result according to its TTL (Time To Live), and returns the IP to the browser.

3. URL Resolution Strategies

- **Absolute URLs:** Include the protocol and domain name (e.g., https://example.com/about.html). Use when linking to external resources on third-party domains.
- **Relative URLs:** Path relative to the current directory (e.g., ../images/photo.jpg). Use for linking assets or pages within closely related directory structures.
- **Root-Relative URLs:** Path starting from the site root directory / (e.g., /assets/css/style.css). Use for site-wide assets or navigation links that need to remain consistent across deep page hierarchies.

4. HTTP Methods: GET vs. POST

- **GET Method:**
    - **Behavior:** Appends data to the URL via query parameters; has size limits; can be cached and bookmarked.
    - **Use Case:** Retrieving data without modifying server state (e.g., searching, reading pages).
- **POST Method:**
    - **Behavior:** Encapsulates data within the HTTP request body; no strict size limits; cannot be cached or bookmarked.
    - **Use Case:** Submitting sensitive data or modifying server state (e.g., form submissions, logins, database writes).

5. HTTPS Security Model

- **Three Guarantees:**
    a. **Confidentiality (Encryption):** Protects data in transit from eavesdropping.

    b. **Data Integrity:** Ensures data cannot be tampered with or corrupted during transmission.
    c. **Authentication:** Verifies the user is communicating with the genuine server associated with the domain name.
- **What it Does NOT Prove:** An HTTPS certificate does **not** prove that the entity behind the website is honest, reputable, or safe from scams—it only proves that communication with that specific domain is secure.

Section 3: Page 67 Practical Reference

Boilerplate HTML Template (template)

```
  <!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Template</title>
</head>
<body>

</body>
</html>
```

Section 4: Practical Programming Solutions

Page 51 Solutions

Basic Questions

1. Create notes.html containing lecture notes with an <h1>, two <h2> sections each with two <h3> subsections and paragraphs, an <hr> between the <h2> sections, one warning, and one emphasis.

Answer:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lecture Notes</title>
</head>
<body>
  <h1>Web Development Lecture Notes</h1>

  <h2>Section 1: Basics</h2>
  <h3>Subsection 1.1: Introduction</h3>
  <p>This section covers basic web development principles.</p>
  <h3>Subsection 1.2: HTML Structure</h3>
  <p>HTML provides the structure for web pages. <em>Always use clean markup.</em></p>

  <hr>

  <h2>Section 2: Security</h2>
  <h3>Subsection 2.1: Input Handling</h3>
  <p><strong>Warning: Never trust user input on the client side alone!</strong></p>
  <h3>Subsection 2.2: HTTPS Protocol</h3>
  <p>HTTPS secures data transmission over the network.</p>
</body>
</html>
```

2. Create gallery.html containing two informative images, one decorative image, and one image link, each with width and height, and place one informative image in a <figure> with a <figcaption>.

Answer:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gallery</title>
</head>
<body>
  <h1>Image Gallery</h1>

  <figure>
    <img src="chart.png" alt="Sales growth chart for 2026" width="400" height="300">
    <figcaption>Figure 1: Annual sales growth report.</figcaption>
```

```
  </figure>

  <img src="diagram.png" alt="System architecture diagram" width="500" height="350">

  <img src="divider.png" alt="" width="600" height="20">

  <a href="index.html">
    <img src="banner.png" alt="Visit Home Page" width="300" height="100">
  </a>
</body>
</html>
```

Intermediate Questions

3. Create contact-card.html using <address> with a telephone link, an email link with a prefilled subject and body, and an external map link that opens in a new tab and says so.

Answer:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contact Card</title>
</head>
<body>
  <h1>Contact Information</h1>
  <address>
    <p>Phone: <a href="tel:+1234567890">+1 (234) 567-890</a></p>
    <p>Email: <a href="mailto:support@example.com?subject=Inquiry&amp;body=Hello%20Support,">Send Email</a></p>
    <p>Location: <a href="map.html" target="_blank" rel="noopener noreferrer">View Location Map (opens in a new tab)</a></p>
  </address>
</body>
</html>
```

4. Create timetable.html containing a weekly timetable with a caption, <thead> and <tbody>, scope on every header cell, a two-hour lab using rowspan, and a public holiday cell spanning a whole day using colspan.

Answer:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Weekly Timetable</title>
```

```
</head>
<body>
  <table>
    <caption>Weekly Class Timetable</caption>
    <thead>
      <tr>
        <th scope="col">Time</th>
        <th scope="col">Monday</th>
        <th scope="col">Tuesday</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <th scope="row">09:00 - 10:00</th>
        <td rowspan="2">Web Dev Lab</td>
        <td colspan="1">Public Holiday</td>
      </tr>
      <tr>
        <th scope="row">10:00 - 11:00</th>
        <td>Lecture</td>
      </tr>
    </tbody>
  </table>
</body>
</html>
```

Advanced Questions

5. Create lecture-video.html with a <video> offering WebM and MP4 sources, a poster, a captions track in a WebVTT file you write, and a fallback download link, followed by a transcript marked up with headings and paragraphs.

Answer:

```
  <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lecture Video</title>
</head>
<body>
  <h1>Lecture Recording</h1>

  <video poster="poster.jpg" controls width="640" height="360">
    <source src="lecture.webm" type="video/webm">
    <source src="lecture.mp4" type="video/mp4">
    <track kind="captions" src="captions.vtt" srclang="en" label="English Captions" default>
    Your browser does not support the video tag.
```

```
    <a href="lecture.mp4">Download lecture video</a>.
  </video>

  <section id="transcript">
    <h2>Video Transcript</h2>
    <h3>Introduction</h3>
    <p>Welcome to modern web development principles.</p>
    <h3>Main Topic</h3>
    <p>Today we will discuss structural HTML markup and accessibility practices.</p>
  </section>
</body>
</html>
```

Challenge Questions

6. Create toc.html containing at least five <h2> sections and a script that builds an ordered list of links to every <h2>, giving each heading an id if it lacks one.

Answer:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Table of Contents</title>
</head>
<body>
  <h1>Document Structure</h1>

  <nav id="toc-container">
    <h2>Table of Contents</h2>
  </nav>

  <h2>Introduction</h2>
  <p>Content for introduction.</p>

  <h2>Architecture</h2>
  <p>Content for architecture.</p>

  <h2>Implementation</h2>
  <p>Content for implementation.</p>

  <h2>Testing</h2>
  <p>Content for testing.</p>

  <h2>Deployment</h2>
  <p>Content for deployment.</p>
```

```
    <script>
        document.addEventListener("DOMContentLoaded", () => {
            const headings = document.querySelectorAll("body > h2");
            const tocContainer = document.getElementById("toc-container");
            const ol = document.createElement("ol");

            headings.forEach((h2, index) => {
                if (h2.parentNode === tocContainer) return;
                if (!h2.id) {
                    h2.id = `heading-${index + 1}`;
                }
                const li = document.createElement("li");
                const a = document.createElement("a");
                a.href = `#${h2.id}`;
                a.textContent = h2.textContent;
                li.appendChild(a);
                ol.appendChild(li);
            });

            tocContainer.appendChild(ol);
        });
    </script>
</body>
</html>
```

Page 68 Solutions

Basic Questions

7. Create symbols.html, which displays the text <html>, the phrase "Terms & Conditions", the copyright sign, the value "100 km" kept on one line, and a check mark written as a hexadecimal reference.

Answer:

```
    <!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Symbols</title>
</head>
<body>
    <p>Text: &lt;html&gt;</p>
    <p>Phrase: Terms &amp; Conditions</p>
    <p>Copyright: &copy;</p>
    <p>Value: 100 km</p>
```

```
  <p>Check mark: &#x2713;</p>
</body>
</html>
```

Intermediate Questions

8. Create languages.html with an English heading and three paragraphs: one in English, one in Arabic marked with lang and dir, and one in French marked with lang. Validate it.

Answer:

```
  <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Languages</title>
</head>
<body>
  <h1>Multilingual Content</h1>
  <p lang="en">This paragraph is written in English.</p>
  <p lang="ar" dir="rtl">هذه الفقرة مكتوبة باللغة العربية.</p>
  <p lang="fr">Ce paragraphe est écrit en français.</p>
</body>
</html>
```

9. Write the HTML document whose body tree is: body containing an h1 ("Library"); a div with id="hours" containing an h2 ("Opening Hours") and a p ("Sunday to Thursday, 08:00 to 22:00"); and a div with id="contact" containing an h2 ("Contact") and a p with text followed by an email link.

Answer:

```
  <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Library</title>
</head>
<body>
  <h1>Library</h1>
  <div id="hours">
    <h2>Opening Hours</h2>
    <p>Sunday to Thursday, 08:00 to 22:00</p>
  </div>
  <div id="contact">
    <h2>Contact</h2>
    <p>For inquiries, please email us at <a href="mailto:info@library.example">info@library.example</a>.</p>
  </div>
```

```
</body>
</html>
```

Advanced Questions

10. Add a Show archive button to Listing 3.2 that toggles the hidden attribute of the archive notice, and a paragraph that displays the data-posted value of the first announcement. Keep the script in its own file.

Answer:

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Archive Toggle</title>
  <script src="script.js" defer></script>
</head>
<body>
  <button id="toggleBtn">Show archive</button>
  <div id="archiveNotice" hidden>
    <p>This is the archived content notice.</p>
  </div>
  <div class="announcement" data-posted="2026-10-23">First Announcement</div>
  <p id="postedDateDisplay"></p>
</body>
</html>
```

JavaScript (script.js):

```
document.addEventListener("DOMContentLoaded", () => {
  const toggleBtn = document.getElementById("toggleBtn");
  const archiveNotice = document.getElementById("archiveNotice");
  const firstAnnouncement = document.querySelector(".announcement");
  const postedDateDisplay = document.getElementById("postedDateDisplay");

  toggleBtn.addEventListener("click", () => {
    archiveNotice.hidden = !archiveNotice.hidden;
  });

  if (firstAnnouncement && postedDateDisplay) {
    postedDateDisplay.textContent = `Posted date: ${firstAnnouncement.dataset.posted}`;
  }
});
```

Challenge Questions

11. Save the source of a public web page, validate it, and fix every error in your copy. Publish a report page with a section for each category of error, each with an id and a link from a summary at the top, showing the error counts before and after.

Answer:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Validation &amp;amp; Error Report</title>
</head>
<body>
  <h1>HTML Validation Report</h1>

  <section id="summary">
    <h2>Summary of Errors</h2>
    <ul>
      <li><a href="#syntax-errors">Syntax Errors</a> - Before: 5, After: 0</li>
      <li><a href="#semantic-errors">Semantic Errors</a> - Before: 3, After: 0</li>
      <li><a href="#attribute-errors">Attribute Errors</a> - Before: 2, After: 0</li>
    </ul>
  </section>

  <section id="syntax-errors">
    <h2>Syntax Errors</h2>
    <p>Fixed unclosed tags and mismatched elements in the original source code.</p>
  </section>

  <section id="semantic-errors">
    <h2>Semantic Errors</h2>
    <p>Replaced non-standard tags with appropriate HTML5 semantic elements.</p>
  </section>

  <section id="attribute-errors">
    <h2>Attribute Errors</h2>
    <p>Added missing required attributes such as alt on img tags and charset meta tag.</p>
  </section>
</body>
</html>
```